

RESTARTING

ESTEGANOGRAFIA E HASH



11

LISTA DE FIGURAS

Figura 1 – Cálculo de uma Função Hash	5
Figura 2 – Criando um arquivo texto no Linux.....	12
Figura 3 – Gerando um Hash MD5 no Linux.....	13
Figura 4 – Site de download do Kali Linux informando o hash de verificação das ISOs	13
Figura 5 – Gerando hash SHA-256 no Linux.....	14
Figura 6 – Gerando hash SHA-512 no Linux.....	15
Figura 7 – Bits menos significativos de uma imagem.....	18
Figura 8 – Bits menos significativos alterados.....	18
Figura 9 – Figura original sem Esteganografia.....	20
Figura 10 – Figura após ser submetida à esteganografia	20
Figura 11 – Processo de Esteganografia com Steghide	22
Figura 12 – Stegsolv com filtro de inversão de cores XOR	23
Figura 13 – Imagem JPG “stringada” sem assinatura de uso do Steghide	24
Figura 14 – Imagem JPG “stringada” com assinatura de uso do Steghide	24
Figura 15 – Criação de conteúdo com o editor nano.....	25
Figura 16 – Esteganografando a imagem	26
Figura 17 – Analisando a imagem.....	26
Figura 18 – Revelando o conteúdo da imagem esteganografada	27
Figura 19 – Assinatura digital	31
Figura 20 – Técnica Pass-the-Key	34
Figura 21 – Técnica Pass-the-Ticket.....	35
Figura 22 – Rainbow Tables.....	40
Figura 23 – Hydra.....	42
Figura 24 – John the Ripper	43
Figura 25 – OphCrack	45
Figura 26 – Metasploit Framework	46
Figura 27 – Saída do Hydra no Kali Linux.....	48

SUMÁRIO

ESTEGANOGRAFIA E HASH.....	4
1 INTRODUÇÃO	4
2 O QUE SÃO AS FUNÇÕES HASH?	4
3 PARA QUE SERVE UMA FUNÇÃO HASH?	6
4 FUNÇÕES HASH MAIS UTILIZADAS	10
4.1 LAB	12
5 ESTEGANOGRAFIA	16
5.1 Entendendo	16
5.1.1 O que é a esteganografia?	16
5.2 Aplicação da esteganografia em imagens.....	17
5.3 Métodos de detecção do uso da esteganografia.....	19
5.4 Ferramentas para esteganografia em imagens.....	21
5.5 Lab	24
6 QUEBRA DE SENHAS.....	27
6.1 Contexto	27
6.2 Criptografia e a função hash	28
6.2.1 Criptografia simétrica ou chave privada	29
6.2.2 Criptografia assimétrica ou chave pública	30
6.3 Assinatura digital	31
6.4 Segurança de senhas no Windows	32
6.5 Ataques <i>Pass-the-Hash</i> e <i>Pass-the-Ticket</i>	33
6.6 Segurança de Senhas no Linux	35
6.7 <i>Wordlists</i>	37
6.8 <i>Rainbow tables</i>	39
6.9 SMB: o vilão dos usuários	40
7 FERRAMENTAS DE QUEBRA DE SENHAS.....	42
7.1 Hydra.....	42
7.2 Medusa.....	43
7.3 John the Ripper	43
7.4 OphCrack	45
7.5 Hashcat	45
7.6 Metasploit Framework	46
7.7 LAB	47
REFERÊNCIAS.....	49

ESTEGANOGRAFIA E HASH

1 INTRODUÇÃO

É comum encontrar quem confunda criptografia, principalmente falando dos princípios de cifragem e encodamento, com função hash. Porém, apesar de visualmente parecidas, são técnicas matemáticas completamente distintas.

As funções hash têm um papel fundamental no campo da Segurança da Informação, pois, por meio delas, é possível garantir princípios, como integridade e confidencialidade às implementações de sistemas informáticos modernos.

Neste capítulo, iremos apresentar os conceitos sobre as funções hash, suas definições matemáticas, as famílias das funções mais utilizadas e ainda as possibilidades de implementação delas em nosso dia a dia.

2 O QUE SÃO AS FUNÇÕES HASH?

Conforme Tavares (2006), as funções criptográficas de hash, ou, simplesmente, funções de hash, possuem diversas características interessantes e que, em um primeiro momento, podem fazer você se perguntar se elas têm alguma utilidade.

De acordo com Serafim (2012), funções de resumo (hash) recebem uma entrada e produzem uma saída, normalmente chamada de hash-code, resultado (ou valor) do resumo ou simplesmente resumo. Uma função de resumo h mapeia cadeias de bits de comprimento arbitrário (embora finito) em cadeias de bits de tamanho fixo (n).

Para um dado domínio D e um contradomínio R , com $h : D \rightarrow R$ e $|D| > |R|$, a função é de muitos para um, o que implica necessariamente a existência de colisões, isto é, valores de $x \neq x'$ tais que $f(x) = f(x')$.

As funções de resumo são utilizadas para muitas aplicações não relacionadas à criptografia (métodos de busca, por exemplo) e, neste escopo, trataremos apenas

das funções de resumo (hash resumo) criptográfico que serão chamadas simplesmente de funções de resumo.

Se considerarmos que o domínio de h consiste de entradas de t bits ($t > n$) e que a função h é aleatória (no sentido de que todas as saídas são igualmente prováveis), então, cerca de 2^{t-n} entradas seriam mapeadas para uma mesma saída e duas entradas escolhidas aleatoriamente geram a mesma saída com a probabilidade 2^{-n} (independente de t).

A ideia é que o valor do resumo serve como uma representação compacta (às vezes chamada de resumo criptográfico, *imprint*, *digital fingerprint* ou *message digest*) da cadeia de entrada e pode ser usado como se identificasse unicamente aquela cadeia.

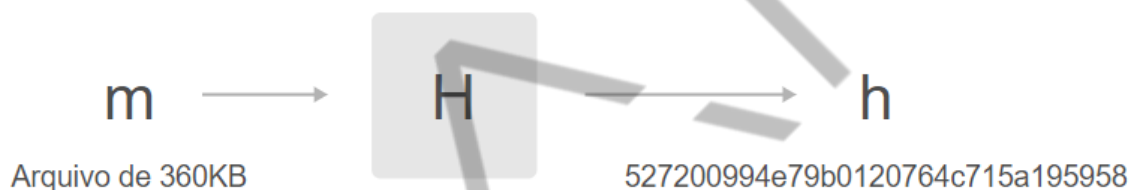


Figura 1 – Cálculo de uma Função Hash
Fonte: Elaborado pelo autor (2019)

Segundo Tavares (2006) e de acordo com a figura acima, a mensagem m é dada como entrada para a função de hash que retorna o valor do hash, ou simplesmente o hash, da mensagem $h = H(m)$.

Uma vez calculado o hash, é impossível, a partir dele, obter-se novamente a mensagem m . Outra característica é o fato de que, não importando o tamanho ou formato da mensagem m , o hash (valor calculado) terá sempre um tamanho fixo em bits (tipicamente 128 bits, 160 bits ou 256 bits). Em outras palavras, se utilizarmos uma função de hash de 128 bits, uma mensagem m de 360 KB, como a do exemplo, ou uma mensagem de 16 GB, elas terão valores de hash com o mesmo tamanho: 128 bits.

Tavares (2006) nos explica que a terceira característica é a resistência a colisões. Se, ao calcularmos o hash de duas mensagens diferentes com uma determinada função de hash e os valores de hash resultantes forem iguais, temos o que é chamado de colisão. Uma boa função criptográfica de hash deve ser resistente

a essas colisões. Por exemplo, consideremos duas mensagens diferentes m_a e m_b , então:

- O hash da mensagem m_a seria: $h_a = H(m_a)$
- hash da mensagem m_b seria: $h_b = H(m_b)$

Se h_a e h_b forem iguais, temos uma colisão.

Ok, entendemos o que é uma colisão. Mas serão elas possíveis? Com certeza, basta analisarmos quantas entradas e saídas possíveis existem para uma função de hash.

Como a entrada é uma mensagem com qualquer formato e qualquer tamanho, podemos considerar que o número de entradas possíveis é infinito (∞). E, observando a saída, se utilizarmos uma função de hash de 128 bits, o número de saídas possíveis é 2^{128} .

Para uma função de hash de 160 bits, os números de saídas possíveis seriam 2^{160} , 2^{128} ou 2^{160} , números muito grandes. Porém, o infinito é... infinito. Sendo assim, podemos afirmar que, para qualquer função de hash, haverá colisões. Portanto, a resistência a colisões não significa que não deve existir colisões, mas sim que, embora colisões existam, elas não podem ser encontradas.

3 PARA QUE SERVE UMA FUNÇÃO HASH?

Conforme Tavares (2006), alguns dos principais usos para as funções hash são:

- Verificação de integridade de arquivos.
- Armazenamento de senhas.
- Resolver o problema da velocidade na assinatura digital.

a) Verificação de integridade de arquivos:

Consideremos o exemplo em que Alice deseja armazenar um arquivo em um servidor e, mais tarde, ao necessitar do arquivo quer ter certeza de que ele continue íntegro.

Alice pode calcular o hash h do seu arquivo m e guardar o hash em algum local seguro:

Alice: $h = H(m)$

Alice: $[h]$

Alice envia, então, apenas o arquivo m para ser armazenado no servidor:

Alice: $m > \text{Servidor}$

Mais tarde, quando precisar do arquivo, Alice recebe o arquivo do servidor e calcula novamente o hash:

Alice: $m < \text{Servidor}$

Alice: $h' = H(m)$

Agora Alice tem dois valores de hash: o valor h que ela havia calculado antes de enviar o arquivo para o servidor e o valor h' que ela calculou depois de recuperar o arquivo do servidor. Assim, basta que Alice compare esses dois valores:

Alice: $i = (h == h')$

Se h for igual à h' , então, a integridade foi mantida, ou seja, o arquivo não sofreu alteração alguma. É exatamente o mesmo arquivo que Alice armazenou anteriormente no servidor.

Porém, se h for diferente de h' , isso irá indicar que a integridade foi comprometida, ou seja, alguém (ou alguma coisa) alterou o arquivo armazenado no servidor. Alice não pode mais confiar no conteúdo do arquivo.

Note que, para essa verificação funcionar, Alice tem que guardar o primeiro hash calculado em algum lugar seguro (ex.: no seu próprio computador). Se ela armazenasse o hash h junto com o arquivo m no servidor, nada impediria que um atacante alterasse o arquivo e, então, calculasse novamente o hash e substituísse o valor calculado pela Alice.

Mais tarde, Alice, ao recuperar o arquivo e verificar a integridade, não perceberia a alteração.

Uma solução para isso é usar um HMAC (*Hash Message Authentication Code*). Um HMAC é, basicamente, uma função de hash com uma chave:

$$h = H(K,m)$$

A ideia é que somente quem conhece a chave K possa gerar hashes válidos. Seguindo com o mesmo exemplo anterior, ao calcular o hash do arquivo, Alice adicionaria uma chave somente de seu conhecimento:

$$\text{Alice: } h = H(K,m)$$

Alice, então, armazena tanto o hash h quanto o arquivo m no servidor:

$$\text{Alice: } m,h > \text{Servidor}$$

Mais tarde, ao recuperar o arquivo do servidor, Alice calcula novamente o hash acrescentando a chave que somente ela conhece:

$$\text{Alice: } m,h < \text{Servidor}$$

$$\text{Alice: } h' = H(K,m)$$

E, então, compara o valor do hash armazenado no servidor com o novo valor de hash calculado:

$$\text{Alice: } i = (h == h')$$

Mais uma vez, se os hashes forem iguais, o arquivo não foi alterado. Caso contrário, os hashes serão diferentes e Alice terá certeza de que houve alguma alteração não autorizada no arquivo e/ou no valor do hash armazenado.

O que muda nesse caso é que um atacante pode alterar o arquivo, mas não consegue gerar um hash que seja igual a um hash calculado pela Alice, pois ele não conhece a chave por ela utilizada.

b) Armazenamento de senhas:

A forma correta de armazenar senhas é com a utilização de funções de hash. Eis a solução correta: o primeiro passo é o cadastramento da senha feito pelo usuário. O usuário informa a sua senha ao sistema:

$$\text{Usuário: } \text{senha} > \text{Sistema}$$

O sistema calcula o hash da senha e o armazena em seu banco de dados:

$$\text{Sistema: } h = H(\text{senha})$$

$$\text{Sistema: } [h]$$

Mais tarde, para autenticar o usuário, o usuário informa a senha ao sistema e o sistema calcula novamente o hash dela.

Usuário: senha > Sistema

Sistema: $h' = H(\text{senha})$

Então, o sistema compara o hash recém-calculado com o hash armazenado para aquele usuário no banco de dados.

Sistema: $i = (h == h')$

Se os hashes forem iguais, significa que o usuário forneceu a mesma senha cadastrada anteriormente. Se forem diferentes, o usuário esqueceu sua senha ou um atacante está tentando se passar por ele.

Essa solução, embora ainda não completa, já fornece uma boa segurança: a senha não é armazenada no banco de dados, mas somente o hash e o hash é irreversível. Assim, nem um atacante externo (*cracker*) nem um interno (administrador ou DBA mal-intencionado) conseguem saber qual é a senha utilizada pelo usuário. Isso é muito importante, já que frequentemente usuários utilizam a mesma senha em diversos sistemas diferentes.

No entanto, ainda temos um detalhe a ser resolvido. Dois usuários que utilizem exatamente a mesma senha, terão, no banco de dados, valores de hash idênticos. É claro que o hash continua sendo irreversível, contudo, agora o atacante pode vir a saber quais usuários utilizam a mesma senha.

Imagine que, em um determinado sistema, Alice e Bob usam a mesma senha 12345, teríamos, então, na tabela de usuários no banco de dados:

alice: 8cb2237d0679ca88db6464eac60da96345513964
bob: 8cb2237d0679ca88db6464eac60da96345513964

A solução é bem simples: salgar o hash. O sal (ou *salt*, do inglês) é alguma informação randômica e não sigilosa juntada à senha antes de calcular o hash. Assim, vamos rever nosso processo.

c) Cadastramento da senha:

Usuário: senha > Sistema

Sistema: $h = H(\text{senha} + \text{sal})$

Agora o sistema gerou um sal (ex.: três caracteres randômicos) e juntou à senha antes de calcular o valor do hash. Mais tarde, o sistema necessitará não só do hash, mas também do sal para autenticar o usuário. Assim, o sistema armazena no banco o hash e o sal do usuário:

Sistema: [h] [sal]

Mais tarde, para autenticar o usuário:

Usuário: senha > Sistema

Sistema: $h' = H(\text{senha} + \text{sal})$

E, mais uma vez, o sistema compara o hash armazenado com o hash recém-calculado para decidir se o usuário é quem ele realmente diz ser. E veja como ficam armazenados os hashes das senhas dos usuários Alice e Bob neste sistema:

alice: 598c39303ddd085d26c4b3a5f478e6f20ec10c25,wrt

bob: 4fc65e108e3573fd690116637953009cd8ca64b3,p8n

Ambos continuam usando a mesma senha, 12345. Todavia, a senha de cada um foi salgada com três caracteres randômicos distintos. Esses três caracteres adicionais alteram completamente o valor do hash e, mesmo não sendo sigilosos, não comprometem em absoluto a irreversibilidade da função de hash utilizada.

4 FUNÇÕES HASH MAIS UTILIZADAS

Iremos abordar as famílias de funções hash mais utilizadas na computação moderna, conforme Serafim (2012).

A família MD5 foi projetada pela RSA (MD2) e por Ron Rivest (MD4 e MD5) e compreende:

- MD2 - RFC 1319.
- MD4 - RFC 1320.
- MD5 - RFC 1321.

Todos os membros da família geram um resumo de 128 bits. Durante muito tempo, o MD5 foi largamente usado, chegando a ser considerado quase um sinônimo de função de resumo. No entanto, devido aos ataques, essa função tem caído em desuso, sendo substituída por outras que têm tamanhos de resumo maiores, como o SHA-1.

Os algoritmos da família SHA (*Secure Hash Algorithms*) foram especificados pelo padrão FIPS (*Federal Information Processing Standards*) 180-2 do NIST US National Institute of Standards and Technology (órgão do governo estadunidense).

O padrão original (FIPS 180-1) especificava apenas o algoritmo conhecido como SHA-1, que tem um resumo (*message digest*) de 160 bits. O FIPS 180-2 (publicado em agosto de 2002) acrescentou três novos algoritmos (SHA-256, SHA-384 e SHA-512) e uma mudança (publicada em fevereiro de 2004) adicionou o algoritmo SHA-224. A RFC 3174 também descreve o SHA-1, trazendo, inclusive, uma implementação em linguagem C do algoritmo.

Os algoritmos da família SHA estão relacionados ao MD5, entretanto, utilizam tamanhos de resumo maiores (a partir de 160). Essa semelhança é, inclusive, fonte de críticas, visto que ataques ao MD5 podem, eventualmente, servir de “entrada” para possíveis ataques às funções da família SHA.

Os algoritmos hoje aceitos pelo padrão são:

- SHA-1.
- SHA-224.
- SHA-256.
- SHA-384.
- SHA-512.

O padrão descreve também uma forma de truncar o resultado para obter resumos de outros comprimentos (por exemplo, 96 bits) como conveniência para facilitar a interoperabilidade de aplicações utilizando tais resumos. No entanto, o padrão não faz nenhuma alegação quanto à segurança de tais resumos truncados e proíbe a utilização deles em aplicações padronizadas que referenciam aquele padrão.

4.1 LAB

Neste *Lab*, iremos usar alguns utilitários de linha de comando Linux para a geração de hash de determinados arquivos. Para tal, utilizaremos nossa VM do Kali Linux. Porém, antes vamos preparar o cenário para a execução do nosso *Lab*.

Abra o terminal e crie um arquivo de texto com o comando abaixo:

```
echo "Aula de funções hash - FIAP" > arquivo.txt
```

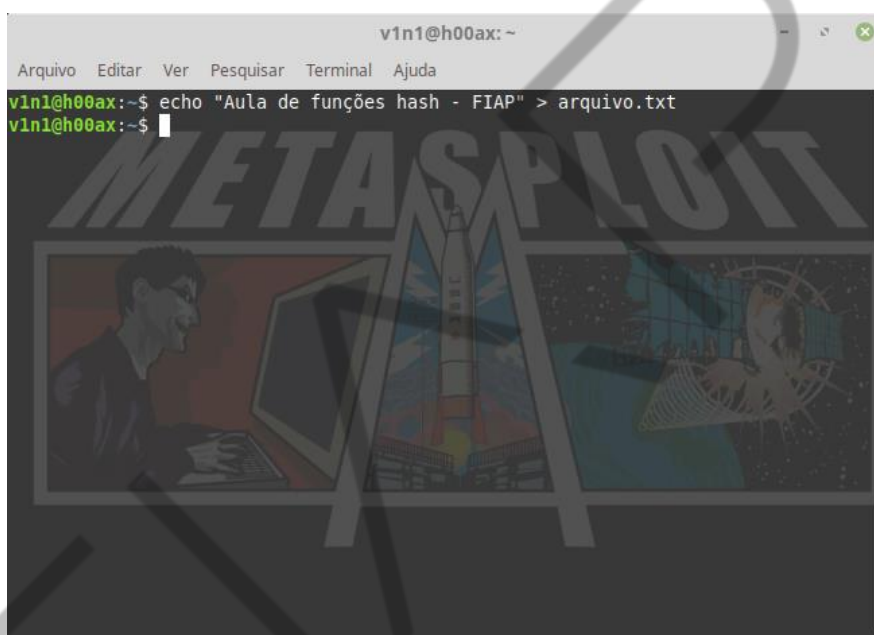


Figura 2 – Criando um arquivo texto no Linux
Fonte: Elaborado pelo autor (2019)

A partir deste momento, já podemos prosseguir na execução do nosso laboratório.

a) Verificando o hash MD5:

Para verificar manualmente o hash MD5 do seu arquivo, basta executar:

```
md5sum arquivo.txt
```

A saída do comando será semelhante a figura “Gerando um Hash MD5 no Linux”:

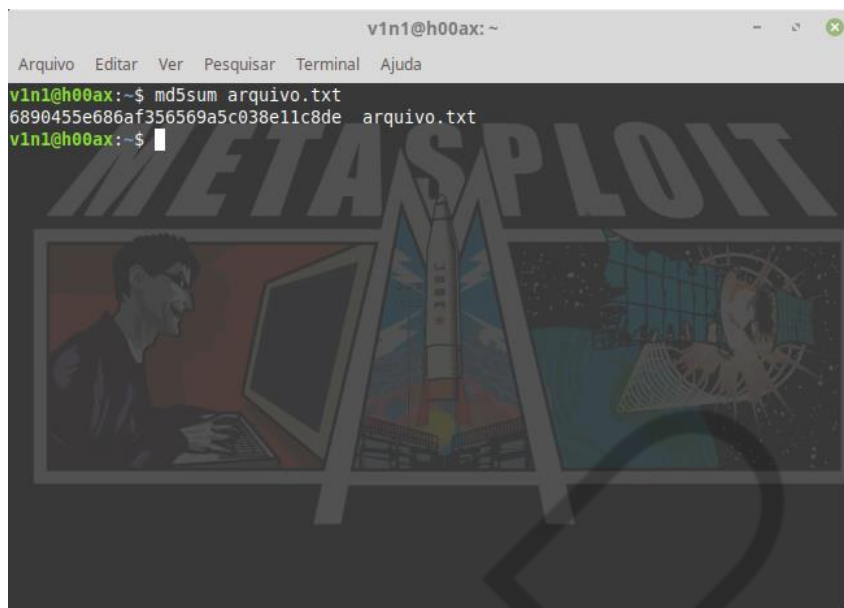


Figura 3 – Gerando um Hash MD5 no Linux
Fonte: Elaborado pelo autor (2019)

Se o arquivo-alvo do *Lab* fosse uma ISO baixada da Internet, por exemplo, você poderia utilizar o `md5sum` para conferir se o download foi realizado sem perda de pacotes. Bastaria, portanto, verificar se o hash gerado era idêntico ao informado no site.

Download Kali Linux Images				
We generate fresh Kali Linux image files every few months, which we make available for download. This page provides the links to download Kali Linux in its latest official release. For a release history, check our Kali Linux Releases page. Please note: You can find unofficial, untested weekly releases at http://cdimage.kali.org/kali-weekly/ . Downloads are rate limited to 5 concurrent connections .				
Image Name	Download	Size	Version	SHA256Sum
Kali Linux Light Armhf	HTTP Torrent	694M	2019.1a	3aaf70621ab0313b14259780d2c50334e23445fc8b27aee30f5b3be94fafad8b
Kali Linux Mate 64 Bit	HTTP Torrent	3.2G	2019.1a	179135fe3abc6ee365558db88e9abc1c32f634c9118d053245390e5c44a9013f

Figura 4 – Site de download do Kali Linux informando o hash de verificação das ISOs
Fonte: Elaborado pelo autor (2019)

Também é possível checar "automaticamente" se o arquivo está ou não corrompido. Para isso, você deverá baixar e salvar o arquivo de verificação MD5 na mesma pasta onde baixou o seu pacote/software/ISO (geralmente disponibilizado na própria página de download); e executar o comando `md5sum` com a opção `-c` (checar):

```
md5sum -c [arquivo-de-verificação]
```

A saída do comando (no meu caso, onde utilizei o pacote LibreOffice) foi a seguinte:

```
$ md5sum -c LibreOffice_5.2.2_Linux_x86-64_rpm.tar.gz.md5
LibreOffice_5.2.2_Linux_x86-64_rpm.tar.gz: SUCESSO
```

Caso apareça a mensagem "SUCESSO" ao rodar o comando acima, indica que o seu arquivo está íntegro, mas se aparecer a mensagem "FALHOU", mostra que o seu arquivo está corrompido. Nesse caso, você terá que baixá-lo novamente.

b) Verificando o hash SHA-256:

O processo é semelhante ao MD5 descrito acima, mudando somente o comando. Para verificar manualmente o hash SHA-256 do seu arquivo, basta executar:

```
sha256sum arquivo.txt
```

A saída do comando será semelhante a esta abaixo:

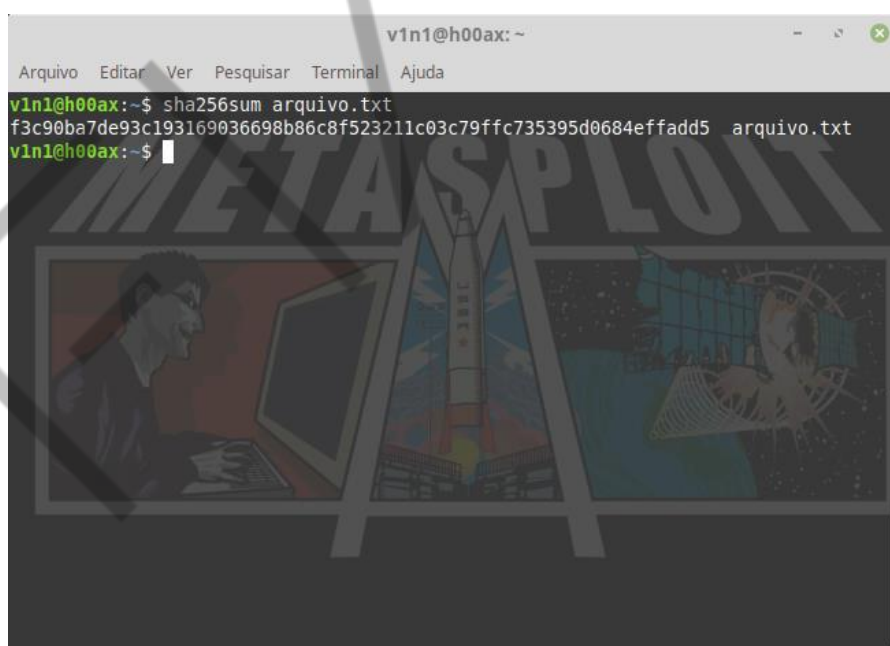


Figura 5 – Gerando hash SHA-256 no Linux
Fonte: Elaborado pelo autor (2019)

Da mesma forma que o MD5, você pode comparar o hash alfanumérico que aparecerá no terminal com o hash SHA-256 disponibilizado no site onde baixou seu arquivo. Se forem iguais, indica que seu arquivo está íntegro, mas se o resultado

obtido no terminal for diferente, o arquivo está corrompido. Nesse caso, você deverá fazer novamente o download do arquivo.

Para verificar "automaticamente" se o arquivo está ou não corrompido, você deverá baixar e salvar o arquivo de verificação SHA-256 na mesma pasta onde baixou o seu pacote/software/ISO, execute o comando sha256sum com a opção -c (checar):

```
sha256sum -c [arquivo-de-verificação]
```

c) Verificando o hash SHA-512:

O processo é semelhante ao SHA-256 descrito acima, mudando somente o comando. Para verificar manualmente o hash SHA-512 do seu arquivo, basta executar:

```
sha512sum arquivo.txt
```

A saída do comando será semelhante a esta abaixo:

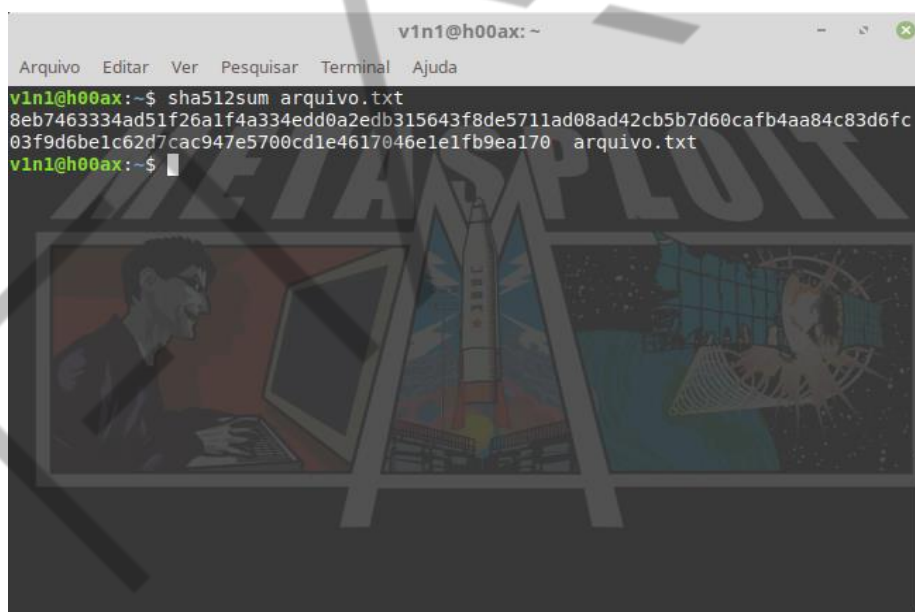


Figura 6 – Gerando hash SHA-512 no Linux
Fonte: Elaborado pelo autor (2019)

Da mesma forma que o SHA-512, você pode comparar o hash alfanumérico que aparecerá no terminal com o hash SHA-512 disponibilizado no site onde baixou seu arquivo. Se forem iguais, isso indica que seu arquivo está íntegro, mas se o resultado obtido no terminal for diferente, o arquivo está corrompido. Nesse caso, você deverá fazer novamente o download do arquivo.

Para verificar "automaticamente" se o arquivo está ou não corrompido, você deverá baixar e salvar o arquivo de verificação SHA-512 na mesma pasta onde baixou o seu pacote/software/ISO, e executar o comando `sha512sum` com a opção `-c` (checar):

```
sha512sum -c [arquivo-de-verificação]
```

5 ESTEGANOGRAFIA

Por diversos momentos, Hollywood tentou demonstrar a vida de espiões em suas "missões impossíveis" ao redor do mundo. Contudo, diferentemente das telas de cinema, em que espiões usam complexos *gadgets* de comunicação criptografada, na realidade, são técnicas simples, como a esteganografia, que permitem o envio e recebimento de mensagens confidenciais entre agentes de Inteligência.

Pare e pense um pouco... você já viu algum perfil no X ou Instagram que, apesar de não ter muitas (ou quase nenhuma postagem), além de ter pouquíssimos seguidores, possui algumas fotos "sem sentido" postadas? Pois é, para você pode não parecer ter muito sentido, mas, com certeza, existe algum destinatário que entende muito bem o conteúdo dessas imagens. E quando digo conteúdo, não me refiro ao que conseguimos ver nessas imagens...

5.1 Entendendo

5.1.1 O que é a esteganografia?

Segundo Coelho e Bento,

esteganografia é uma palavra de origem grega, em que *Stegano* significa escondido ou secreto e *Grafia*, escrita ou desenho. Diferente da criptografia, que esconde o conteúdo de mensagem, tornando-a ilegível, a Esteganografia obscurece a existência de informação, ocultando-a (Coelho; Bento, 2004 apud Martins *et al.*, 2017, p. 2).

Uma mensagem oculta por técnicas esteganográficas não atrai atenção a si mesma, ao transmissor ou ao receptor. Uma das formas básicas entre as técnicas de esteganografia está na alteração do bit menos significativo, que não afeta a visualização da imagem final.

Ainda conforme Martins *et al.* (2017), a técnica de esteganografia tem sido utilizada desde a Antiguidade, quando se desejava encaminhar mensagens escondidas. A primeira utilização da técnica confirmada pode ser encontrada em *As Histórias*, de Heródoto, descrevendo, no século V a.C., que um tirano grego foi aprisionado por um rei.

O tirano queria fazer contato secreto com seu superior, escolhendo um escravo fiel que fizesse o canal. Ele teve sua cabeça raspada e em seu couro cabeludo foi feita uma tatuagem com uma mensagem para ser enviada ao superior. O fiel aguardou seu cabelo crescer e, posteriormente, foi enviado com a instrução de raspar novamente seus cabelos para que a mensagem, enfim, fosse lida.

Com o desenvolvimento da tecnologia, a esteganografia foi aplicada no ambiente computacional a fim de mascarar dados sem detecção do humano.

Um exemplo básico da aplicação da técnica é a inserção do dado em um objeto de cobertura, que produz a ocultação. Logo após, a estego-chave controla todo o processo, restringindo a recuperação e/ou a detecção do dado inserido.

Porém, assim como foram desenvolvidas ferramentas para criar a camuflagem, existe a esteganálise, que recupera textos, imagens e vídeos ocultados em um objeto de cobertura.

5.2 Aplicação da esteganografia em imagens

De acordo com Martins *et al.* (2017), a utilização de esteganografia com o auxílio de softwares abrange diversos métodos para a ocultação de dados. São eles:

a) Conforme Rocha (2003), *Least Significant Bit* (LSB) se baseia em modificar os bits menos significativos de cada pixel (menor unidade da imagem digital), camuflando, assim, as informações. A forma de inserir dados com o LSB seria da seguinte maneira representada na figura abaixo.

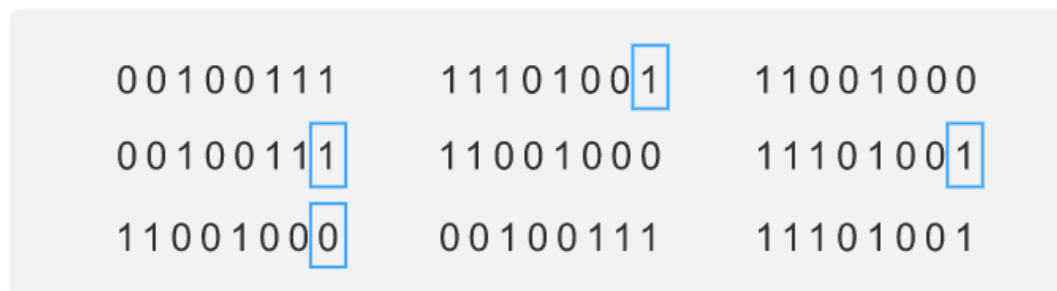


Figura 7 – Bits menos significativos de uma imagem
Fonte: Google Imagens (2019)

Com a introdução do valor binário 10000011 da mensagem em 9 pixels, iniciando do byte da esquerda superior, o resultado seria como está demonstrado na figura a seguir.

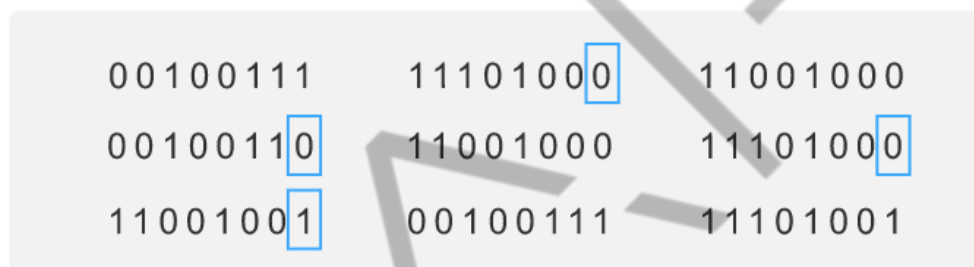


Figura 8 – Bits menos significativos alterados
Fonte: Google Imagens (2019)

b) Martins *et al.* (2017) comentam que a filtragem e o mascaramento modificam os bits mais significativos de imagens, ao contrário do LSB, que utiliza os bits menos significativos e com fácil implementação. Por modificar os bits mais significativos, acaba tornando mais suscetível à detecção, mas imperceptível a olho nu pelo ser humano.

c) Em algoritmos e transformações, as informações são inseridas em áreas mais robustas que estão espalhadas pela imagem inteira, obtendo melhor resistência ao processar o sinal. As imagens no formato .jpeg utilizam a compressão em que há perda de dados *Discrete Cosine Transform* (DCT). Isso porque os valores cosseno são calculados imprecisamente, ocasionando a introdução de arredondamento dos erros.

Conforme Martins *et al.* (2017), a técnica baseada em algoritmos e transformações, tem como vantagem a compressão, um dos principais problemas encontrados no LSB. Essa técnica utiliza a transformada de Fourier discreta, transformada de cosseno discreta e transformada Z.

Os dados ocultos são inseridos em áreas mais robustas que estão alocadas em toda a imagem, favorecendo resistência maior contra o processamento de sinal. Contudo, mesmo havendo técnicas sofisticadas, não quer dizer que implicará melhor robustez para ataques de esteganálise.

d) A técnica de espalhamento de espectro distribui os dados ocultos na imagem cobertura, utilizando uma chave-estego para a seleção randômica dos canais de frequência. A distribuição dos dados inseridos ocorre com a modulação deles por meio de pseudoruído. Após o processo, a energia é distribuída em uma larga faixa de frequência que alcança apenas um nível muito inferior da força de inserção, garantindo a imperceptibilidade.

5.3 Métodos de detecção do uso da esteganografia

De acordo com Azevedo *et al.* (2015), a esteganálise – o estudo analítico de arquivos possivelmente esteganografados – consiste na ideia de detectar uma mensagem oculta via esteganografia. Esse tipo de processo atua de duas formas: a primeira tem como objetivo identificar a presença de mensagens ocultas na imagem; a outra, extrair da imagem a mensagem oculta.

De acordo com Albuquerque (2007) apud Azevedo *et al.* (2015), os tipos de esteganálise são:

a) Ataques aurais: Retiram partes significativas do objeto de cobertura como um meio de facilitar a busca por ruídos adicionados via esteganografia.

b) Ataques estruturais: Visam procurar alterações no padrão do arquivo do objeto de cobertura.

c) Ataques estatísticos: Procuram encontrar padrões de comportamento do conteúdo do arquivo. Nos dias de hoje, dificilmente são encontradas aplicações técnicas de ataques para mídias contínuas, ou seja, áudio e vídeo. O foco principal refere-se à esteganálise em imagens.

No estudo feito por Azevedo *et al.* (2015), foi utilizada uma ferramenta de esteganografia que aceitava imagens do tipo “gif”, “jpeg”, “jpg” e “png” e foram feitos testes em cada uma das extensões. Após isso, por meio do terminal Linux, comparou-

se o tamanho do arquivo original e o esteganografado. Como exemplo, foi embutida na figura abaixo a frase:

Definições, assim como perguntas e metáforas, são instrumentos para fazer pensar. Sua autoridade reside inteiramente na sua utilidade, não na sua correção. Usamos definições a fim de delinear o problema que desejamos investigar ou favorecer interesses que queremos promover. Em outras palavras, inventamos definições e as descartamos na medida em que servem aos nossos propósitos (Postman, 1980, p. 25 *apud* Azevedo *et al.*, 2015, p. 34).



Figura 9 – Figura original sem Esteganografia
Fonte: Google Imagens (2019)

Após o processo, podemos comparar a “Figura original sem Esteganografia” com a “Figura após ser submetida à esteganografia”, notando que não há nenhuma diferença em nenhum dos cinco parâmetros analisados. Isso ocorre por causa do método LSB, que “esconde” a mensagem, possibilitando seu envio de forma mais segura.



Figura 10 – Figura após ser submetida à esteganografia
Fonte: Azevedo *et al.* (2015)

No experimento, foram esteganografadas 10 figuras de cada uma das extensões (quatro tipos). As amostras possibilitaram obter um comparativo com os arquivos originais. Os critérios, como formato da figura, qualidade e cor, foram analisados visualmente. Já o tamanho dos pixels e do arquivo foi observado por meio do terminal Linux e foram consultadas as suas propriedades.

A “Figura após ser submetida à Esteganografia” demonstra a imagem esteganografada, propiciando, assim, observar que, apesar de possuir uma mensagem em seu interior, não houve alterações visuais perceptíveis. Observou-se o mesmo comportamento em todas as extensões. Nenhuma das extensões apresentou qualquer tipo de alteração na figura que permitisse algum interceptador encontrar a mensagem embutida.

5.4 Ferramentas para esteganografia em imagens

Tomando como base a distribuição Kali Linux, utilizada em nossas aulas por ter seu foco em Segurança Cibernética, apresentaremos a ferramenta de terminal Steghide, utilizada para esteganografia de imagens.

De acordo com Azevedo *et al.* (2015), o Steghide é um programa de esteganografia capaz de esconder dados em vários tipos de arquivos de áudio e de imagem. As frequências de som e de cor, respectivamente, não são alteradas, tornando o arquivo resistente contra testes estatísticos de primeira ordem.

O algoritmo de criptografia-padrão é o Rijndael com uma chave de 128 bits de comprimento, no qual os “dados secretos” são compactados e criptografados. Em seguida, uma sequência de posições de pixels no arquivo que servirá de “esconderijo” é criada com base em um gerador de números aleatórios inicializado com a senha (os “dados secretos” serão incorporados nos pixels dessas posições).

Posteriormente, um algoritmo de correspondência grafoteórico encontra pares de posições que troquem seus valores. Isso tem o efeito de incorporar a parte correspondente dos “dados secretos”. Se o algoritmo não consegue encontrar mais pares, todas as trocas são efetivamente realizadas.

O Steghide usa o método LSB, ou seja, substitui o bit menos significativo dos pixels que formam as cores das imagens do tipo Bitmap – BMP – em imagens desse

tipo, as cores nos pixels são formadas da seguinte maneira: imagens como “jpg” possuem blocos sucessivos de 8 bits, o que permite que se tenha 224 cores diferentes (por volta de 16 milhões de tonalidades), com 24 bits por pixel.

O método de Esteganografia LSB vai substituir o bit menos significativo de cada uma das três cores que formam um pixel (RGB – *Red* (Vermelho), *Green* (Verde) e *Blue* (Azul)). Assim, cada pixel aceita até 3 bits de informação. Desse modo, cada imagem consegue armazenar até 3 vezes o número de pixels que possui, assim como demonstrado na figura abaixo.

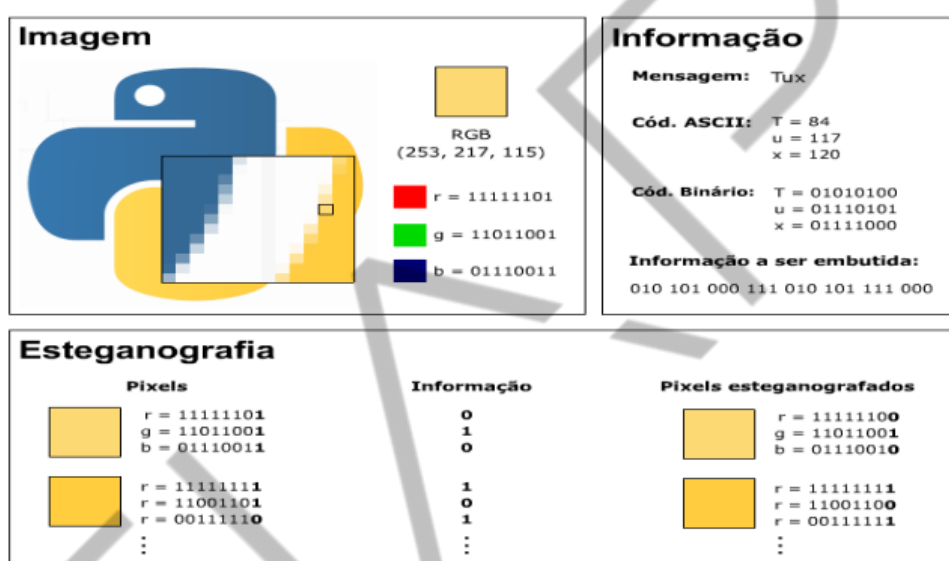


Figura 11 – Processo de Esteganografia com Steghide
Fonte: Google Imagens (2019)

Outra ferramenta interessante, apesar de não vir por padrão no Kali Linux, é o Stegsolv. Escrito em Java, o Stegsolv foi desenvolvido para ataques a imagens, em que aplica diversos filtros, como, por exemplo, inversão de cores (XOR), para que o analista consiga visualmente identificar conteúdos escondidos em imagens por meio de esteganografia.

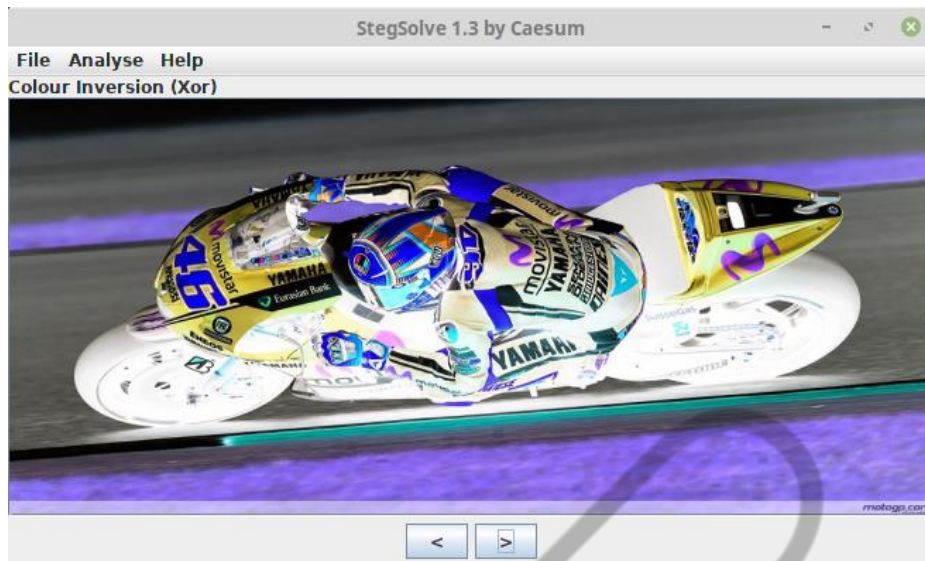


Figura 12 – Stegsolv com filtro de inversão de cores XOR
Fonte: Elaborado pelo autor (2019)

Ferramentas de esteganografia, a exemplo do Steghide, além de embutirem conteúdos em imagens, conseguem protegê-los da decifragem por meio de senhas.

Logo, em uma estegoanálise na qual se identifique a existência de conteúdo esteganografado em imagem, porém não se consiga extrair devido à necessidade de senha, existem ferramentas como o StegCrack que conseguem automatizar um ataque de dicionário (*password guessing*) na imagem esteganografada.

Quanto à detecção da esteganografia por meio de ataques estruturais, podemos citar ferramentas como o StegDetec, que, aplicado à imagem analisada, traz informações quanto ao possível conteúdo esteganografado.

Porém, no caso de a esteganografia ter sido feita por meio do Steghide, uma forma simples de se identificar é analisando a imagem por meio do comando *strings* (presente no Kali Linux), tendo em vista que o Steghide deixa um padrão de assinatura na imagem esteganografada, conforme as figuras “Imagem JPG ‘stringada’ sem assinatura de uso do Steghide” e “Imagem JPG ‘stringada’ com assinatura de uso do Steghide”.

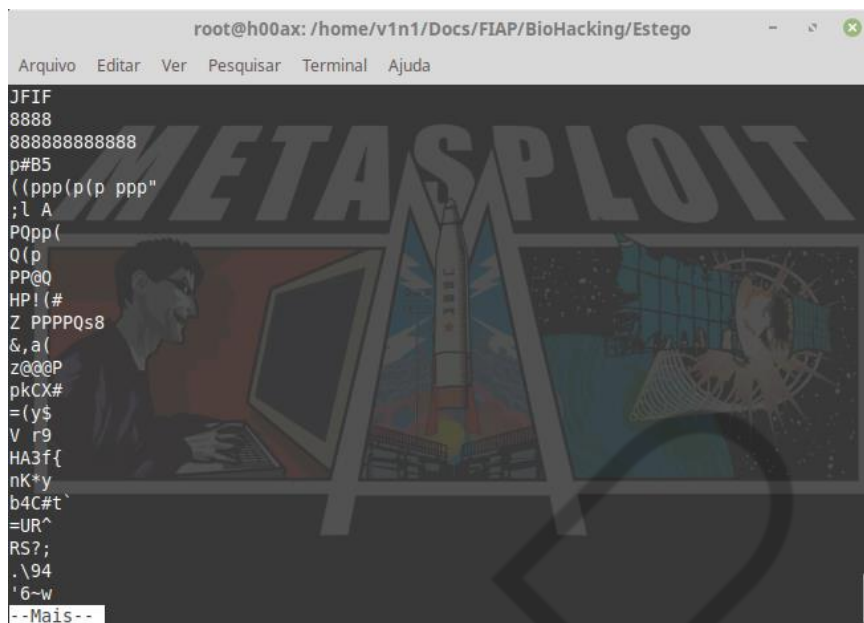


Figura 13 – Imagem JPG “stringada” sem assinatura de uso do Steghide
Fonte: Elaborado pelo autor (2019)

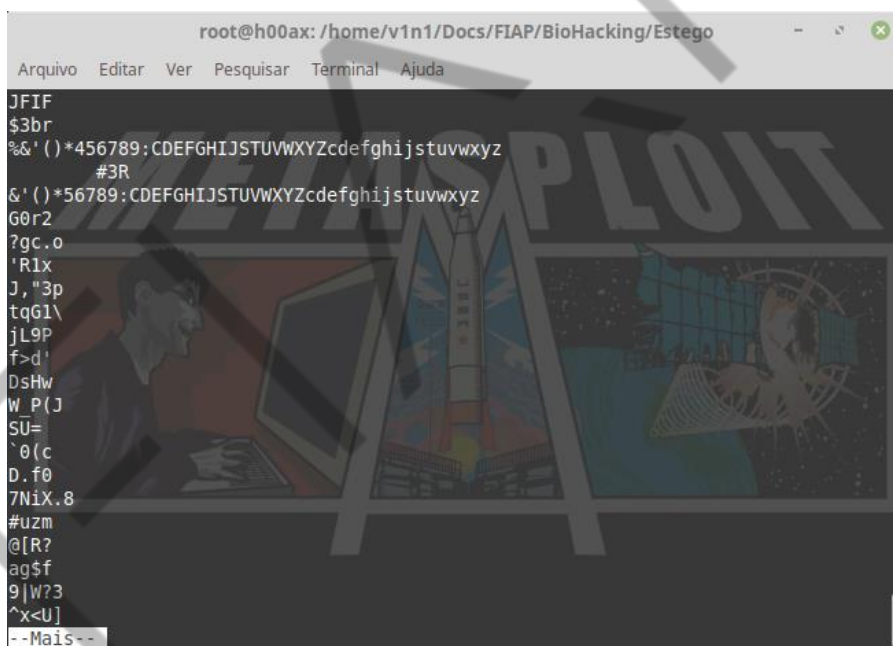


Figura 14 – Imagem JPG “stringada” com assinatura de uso do Steghide
Fonte: Elaborado pelo autor (2019)

5.5 Lab

Neste Lab, iremos esteganografar um conteúdo em uma imagem, utilizando o Kali Linux e a ferramenta Steghide.

Baixe a imagem que achar mais indicada para sua máquina virtual do Kali Linux e execute os procedimentos abaixo:

a) Preparação do conteúdo que será embutido na imagem:

Abra o editor de textos nano pelo terminal do seu Kali e crie o arquivo “fiap.txt” com o seguinte conteúdo:

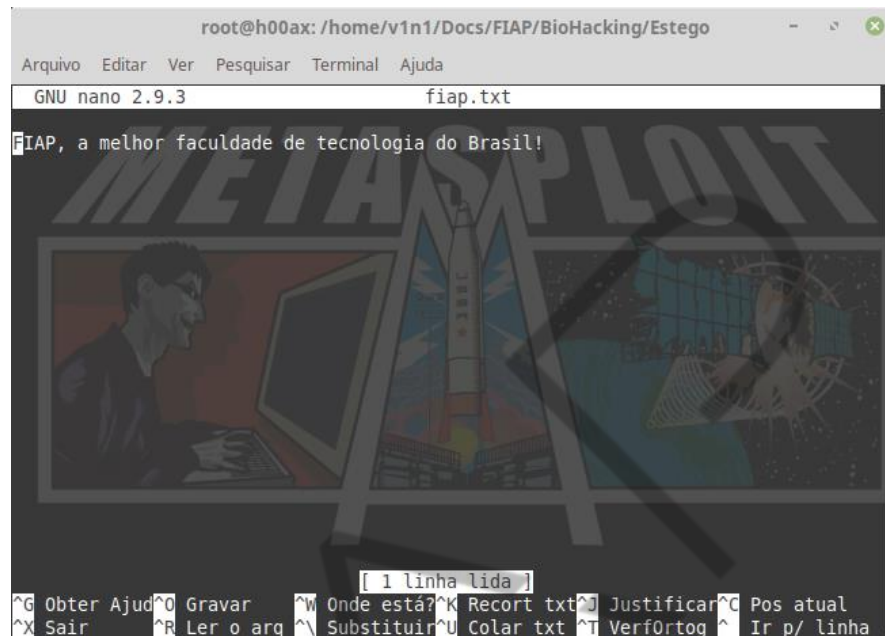


Figura 15 – Criação de conteúdo com o editor nano
Fonte: Elaborado pelo autor (2019)

Pressione Ctrl+O para salvar e depois Ctrl+X para sair do arquivo.

b) Esteganografando a imagem:

Utilizando o Steghide, iremos embutir o arquivo “fiap.txt” na nossa imagem modelo, conforme a sintaxe abaixo (escolha uma senha de sua preferência):

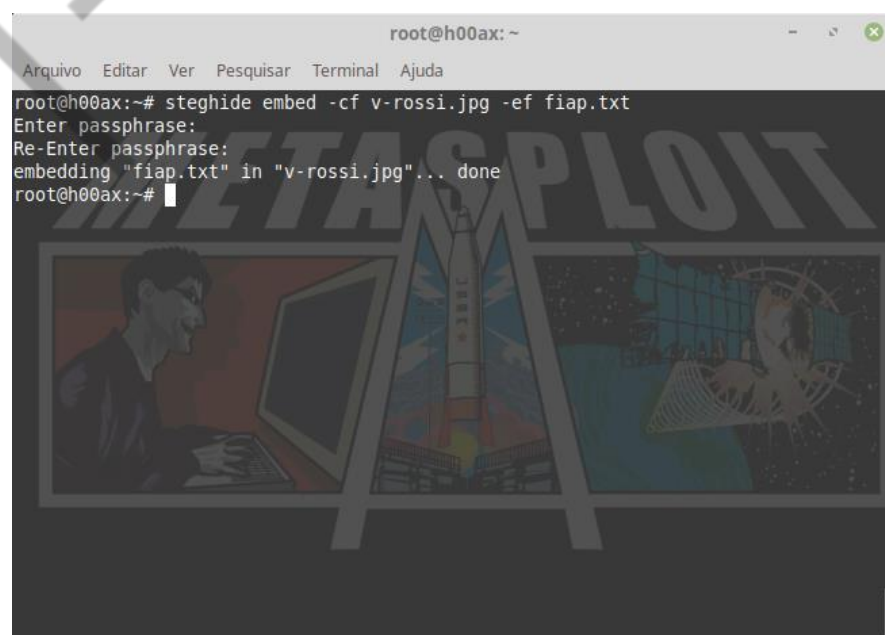


Figura 16 – Esteganografando a imagem
Fonte: Elaborado pelo autor (2019)

c) Conferindo a assinatura do Steghide:

Usando o *strings*, iremos analisar a imagem modelo para identificar o padrão de assinatura do Steghide, com a sintaxe abaixo (para sair do prompt do strings, basta pressionar a tecla “q”):

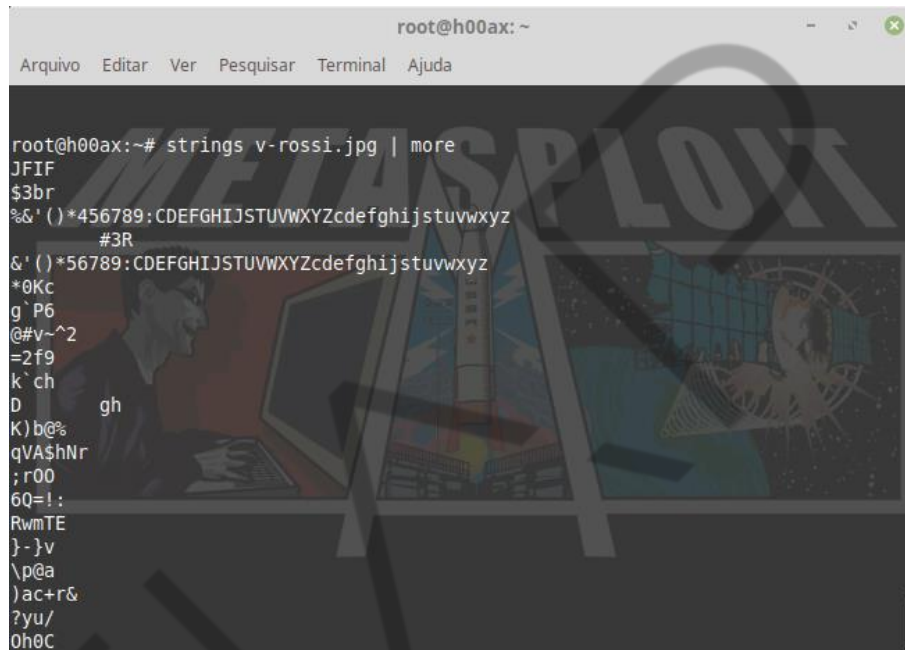


Figura 17 – Analisando a imagem
Fonte: Elaborado pelo autor (2019)

d) Revelando o conteúdo esteganografado da imagem:

Iremos utilizar a sintaxe abaixo do Steghide para exibir o conteúdo esteganografado da imagem. Antes disso, para que o arquivo de saída não se confunda com o arquivo “fiap.txt” que está no diretório corrente, crie um subdiretório e mova a imagem modelo para lá antes de utilizar o Steghide para remover a esteganografia, desta forma:

```
mkdir estego  
mv v-rossi.jpg estego/  
cd estego
```

Agora sim, utilize o Steghide para revelar o conteúdo da imagem modelo.

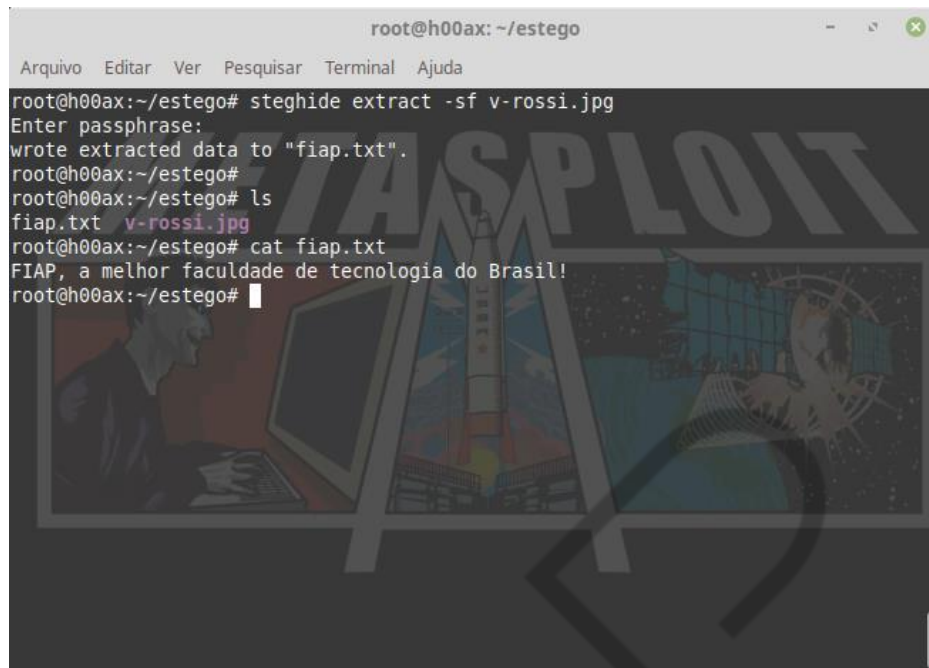


Figura 18 – Revelando o conteúdo da imagem esteganografada
Fonte: Elaborado pelo autor (2019)

6 QUEBRA DE SENHAS

Um dos princípios da Segurança da Informação é a autenticidade, e é sob a égide desse princípio que usamos comumente logins e senhas nos diversos sistemas que acessamos diariamente, pois entende-se que, por meio da identificação do usuário pelo login e senha, se confirma que ele é quem diz ser.

6.1 Contexto

Filosoficamente, esse conceito é muito interessante. Porém, na prática, é infundado, visto que a identificação pelo login e senha não prova que aquela identidade foi realmente utilizada pelo seu detentor. Seria algo muito parecido como em um furto de um carro em que o criminoso obteve as chaves antes de roubá-lo.

De posse delas, ele não precisou violar os sistemas de segurança do carro, ou seja, ele não é o dono do carro nem das chaves, mas conseguiu acesso a ele de forma legítima utilizando a chave correta.

Entender o funcionamento de um ataque de força bruta, no qual se acessa determinado alvo com as credenciais legítimas sem tê-las de antemão, por meio de diversos métodos, é fundamental para um profissional de Defesa Cibernética.

Não só do ponto de vista do ataque, haja vista que esse profissional irá realizar diversos testes de penetração em sistemas e redes e precisará entender bem esses conceitos, mas também do ponto de vista da defesa. Visto que, por meio dos conhecimentos adquiridos nesta disciplina, será possível implementar medidas de defesa contra ataques de força bruta, sejam elas por meio de tecnologias ou de políticas e conscientização em sua organização.

6.2 Criptografia e a função hash

A fim de entendermos a concepção de um processo de autenticação que utiliza uma senha para determinado sistema computacional, precisamos antes estabelecer os conceitos basilares sobre criptografia e função hash, além de relembrar alguns fundamentos da Segurança da Informação.

Sob esse ponto de vista, conforme Oliveira (2012), a credibilidade nos sistemas computacionais é definida pela disponibilidade, integridade, controle de acesso, autenticidade, não repudição e, finalmente, a privacidade, que devem ser de livre compreensão e facilmente perceptíveis ao se efetuar transações computacionais.

- **Disponibilidade** – Certificar que uma informação estará disponível para acesso no momento desejado.
- **Integridade** – Garantir que o conteúdo da mensagem não foi alterado.
- **Controle de acesso** – Assegurar que o conteúdo da mensagem somente será acessado por pessoas autorizadas.
- **Autenticidade** – Atestar a identidade de quem está enviando a mensagem.
- **Não repudição** – Prevenir que alguém negue o envio e/ou recebimento de uma mensagem.
- **Privacidade** – Impedir que pessoas não autorizadas tenham acesso ao conteúdo da mensagem, garantindo que apenas a origem e o destino tenham conhecimento.

Assim é fundamental que técnicas computacionais sejam empregadas para que os requisitos de proteção da informação sejam atendidos. Nesse cenário, apresentam-se os dois tipos básicos de criptografia: a simétrica ou chave privada, e a assimétrica ou chave pública.

6.2.1 Criptografia simétrica ou chave privada

Segundo Oliveira (2012), o modelo mais antigo de criptografia é o da chave, isto é, o elemento que dá acesso à mensagem oculta trocada entre duas partes, que é igual (simétrica) para ambas as partes e deve permanecer em segredo (privada). Tipicamente, essa chave é representada por uma senha, usada tanto pelo remetente para codificar a mensagem numa ponta como pelo destinatário para decodificá-la na outra.

Essencialmente, quando a origem (ALFA) cifra uma mensagem, ela utiliza um algoritmo de ciframento para transformar o conteúdo claro da mensagem em texto cifrado.

Quando o destino (BRAVO) decifra uma mensagem, ele utiliza o algoritmo de deciframento correspondente para converter o texto cifrado de novo em uma mensagem clara.

Se um intruso (CHARLIE) conhecer o algoritmo de ciframento, ele poderá decifrar uma mensagem cifrada tão facilmente quanto o destino (BRAVO). A solução no uso da criptografia de chave privada propõe que quando a origem (ALFA) cifra uma mensagem, ela utilize um algoritmo de ciframento e uma chave secreta para transformar uma mensagem clara em um texto cifrado.

O destino (BRAVO), por sua vez, ao decifrar a mensagem, utiliza o algoritmo de deciframento correspondente e a mesma chave para transformar o texto cifrado em uma mensagem em claro.

O intruso (CHARLIE), por não possuir a chave secreta, mesmo conhecendo o algoritmo, não conseguirá decifrar a mensagem. A segurança do sistema passa a residir não mais no algoritmo e, sim, na chave empregada. É ela (chave privada) que agora, no lugar do algoritmo, deverá ser mantida em segredo pela origem (ALFA) e destino (BRAVO).

6.2.2 Criptografia assimétrica ou chave pública

De acordo com Oliveira (2012), esse é um modelo de criptografia criado na década de 1970 – pelo matemático Clifford Cocks, que trabalhava no serviço secreto inglês, o GCHQ – no qual cada parte envolvida na comunicação usa duas chaves diferentes (assimétricas) e complementares, uma privada e outra pública.

Nesse caso, as chaves não são apenas senhas, no entanto, arquivos digitais mais complexos (que eventualmente até estão associados a uma senha). A chave pública pode ficar disponível para qualquer pessoa que queira se comunicar com outra de modo seguro, mas a chave privada deverá ficar em poder apenas de cada titular. É com a chave privada que o destinatário poderá decodificar uma mensagem que foi criptografada para ele com sua respectiva chave pública.

Para entender o conceito, basta pensar num cadeado comum protegendo um determinado bem. A mensagem é esse bem, e o cadeado, que pode ficar exposto, é a chave pública. Apenas quem tiver uma chave particular (privada) que consiga abrir o cadeado poderá acessar a mensagem.

A principal vantagem desse método é a sua segurança, uma vez que não é preciso (nem se deve) compartilhar a chave privada. Por outro lado, o tempo de processamento de mensagens com criptografia assimétrica é muitas vezes maior do que com criptografia simétrica, o que pode limitar seu uso em determinadas situações.

Essencialmente, o destino (BRAVO) e todos os que desejam comunicar-se de modo seguro geram uma chave de ciframento e sua correspondente chave de deciframento. Ele mantém secreta a chave de deciframento, que é chamada de sua chave privada. Ele torna pública a chave de ciframento, que é chamada de sua chave pública. A chave pública realmente condiz com seu nome. Qualquer pessoa pode obter uma cópia dela. O destino (BRAVO), inclusive, encoraja isso, enviando-a para seus amigos ou publicando-a na Internet.

Assim, o intruso (CHARLIE) não tem nenhuma dificuldade em obtê-la. Quando a origem (ALFA) deseja enviar uma mensagem ao destino (BRAVO), precisa primeiro encontrar a chave pública dele. Feito isso, ela cifra sua mensagem utilizando a chave pública do destino (BRAVO), despachando-a em seguida.

Quando o destino (BRAVO) recebe a mensagem, ele a decifra facilmente com sua chave privada. O intruso (CHARLIE), que interceptou a mensagem em trânsito, não conhece a chave privada do destino (BRAVO), embora conheça sua chave pública. Mas esse conhecimento não o ajuda a decifrar a mensagem. Mesmo a origem (ALFA), que foi quem cifrou a mensagem com a chave pública do destino (BRAVO), não pode decifrá-la agora.

6.3 Assinatura digital



Figura 19 – Assinatura digital
Fonte: Google Imagens (2019)

Segundo Oliveira (2012), o sistema de criptografia assimétrica ou de chave pública também é utilizado como um meio de assinatura digital. A pessoa que assina usa sua chave privada para criptografar uma mensagem conhecida, e o texto cifrado pode ser decifrado por qualquer um usando a chave pública dessa pessoa. Assim como uma assinatura em papel, consiste em um bloco de informação adicionado à mensagem que comprova a identidade do emissor, confirmando quem ele diz ser.

O processo se baseia em uma inversão do sistema, em que o funcionamento da assinatura digital pode ser descrito como: o emissor cifra (ou seja, atesta autenticidade) a mensagem com sua chave privada e a envia, em um processo denominado de assinatura digital.

Cada um que receber a mensagem deverá verificar a validade da assinatura digital, utilizando, para isso, a chave pública do emissor, reconhecendo, de fato, que a mensagem não foi adulterada. Como a chave pública do emissor apenas decifra (ou seja, verifica a validade) mensagens cifradas com sua chave privada, obtém-se a garantia de autenticidade, integridade e não repudição da mensagem, o que é

apoiado pela função *hashing*. Se alguém modificar um bit do conteúdo da mensagem ou se outra pessoa assiná-la em vez do próprio emissor, o sistema de verificação não irá reconhecer a assinatura digital dele como sendo válida.

6.4 Segurança de senhas no Windows

De acordo com Moraes (2016), as credenciais do Windows podem ser armazenadas de diferentes formas, porém a mais comum (e padrão dos sistemas Windows desde o Windows NT) é a base de dados Security Account Manager (SAM).

Um componente chamado LSA secrets armazena cache de credenciais do domínio que são usadas pela Local Security Authority (LSA). É uma espécie de armazenamento protegido. A LSA é responsável, entre outras coisas, pelo logon de usuários no sistema.

Credenciais podem ser armazenadas na memória do computador por meio do processo LSASS. As senhas podem ser armazenadas usando hashes LM ou NTLM.

Cabe ressaltar que o hash NTLM é a mesma chave de entrada para o tipo de criptografia RC4-HMAC, o qual é um hash único e não muda independentemente da senha escolhida. Apesar de ser um grave erro de segurança, a Microsoft aparentemente implementou dessa forma por questões de compatibilidade com sistemas Windows mais antigos.

LSASS é um acrônimo que vem de *Local Security Authority Subsystem Service* (serviço de subsistema de autoridade de segurança local). Nesse caso, pode haver armazenadas senhas com criptografia reversível (*reversibly encrypted passwords*), hashes de senhas (NT ou LM) ou tíquetes do Kerberos (TGT e outros tíquetes de serviço).

Hashes também podem ser achadas na base de dados dos controladores de domínio do AD, que retêm todas as credenciais do domínio, por acesso via serviços de diretórios ou no arquivo NTDS.DIT.

Há também a possibilidade de que credenciais de domínio sejam armazenadas localmente de modo que usuários possam fazer logon de forma interativa mesmo se o controlador de domínio não estiver disponível. É o caso de laptops e notebooks que são colocados como participantes de um domínio e precisam que seus usuários sejam

capazes de fazer logon mesmo que não estejam na estrutura do trabalho. Esse mecanismo é conhecido como *cached credentials*.

Esses são alguns exemplos de onde credenciais podem ser furtadas. Esses locais são visados por atacantes que buscam extrair credenciais. Atacantes, geralmente, após comprometer um sistema, escalam privilégios e tentam obter credenciais desses locais, procurando no disco rígido ou com alguma ferramenta que leia a memória ou faça despejo dela.

6.5 Ataques *Pass-the-Hash* e *Pass-the-Ticket*

Ainda conforme Moraes (2016), *Pass-the-Hash* (PtH) e *Pass-the-Ticket* (PtT) são técnicas que utilizam um hash de senha, ou um tíquete do Kerberos, para personificar o usuário que detém a senha para aquele hash ou o tíquete.

Nesses casos, tanto o hash como o tíquete funcionam como equivalentes de senha. O protocolo Kerberos especifica operações a serem aplicadas na senha de usuário para transformar uma senha de tamanho variável em uma chave de longo prazo de tamanho fixo, que servirá como entrada para as várias formas de algoritmos de criptografia suportados pela especificação (AES 128, AES 256, DES, 3DES, RC4...).

As chaves geradas podem ser armazenadas no processo LSASS para serem usadas pelo usuário durante a sessão (sessão LSA). Como essas chaves servem como entrada para os diferentes tipos de algoritmos de criptografia, elas são usadas nesse sentido quando furtadas, como equivalentes de senha. Há várias ferramentas para executar esses ataques. No Kali Linux, temos, por exemplo, o Mimikatz, Psh-toolkit, Msvctl, Metasploit PSEXEC module etc.

Moraes (2016) nos traz que um modelo de autenticação-padrão, largamente conhecido, é aquele em que o servidor armazena em sua base de dados os hashes da senha de usuário. Isso é feito para que, caso a base de dados seja comprometida, o atacante não tenha acesso imediato ao texto claro das senhas de usuários.

Em uma arquitetura cliente-servidor, em um processo de autenticação, é comum o cliente passar a senha para o servidor por meio de um canal seguro

preestabelecido. O servidor calcula o hash da senha e compara-o com o armazenado em sua base de dados, autenticando o usuário caso os hashes coincidam.

Imagine uma aplicação em que o hash da senha de usuário seja calculado no cliente e enviado por meio de um canal inseguro para o servidor, que comparará os hashes e o autenticará caso coincidam. Um atacante capturando o tráfego na rede pode obter esse hash e repetir a chamada do cliente ao servidor, personificando o usuário e obtendo a autenticação do servidor. Esse exemplo é o tipo de ataque conhecido como *Pass-the-Hash*.

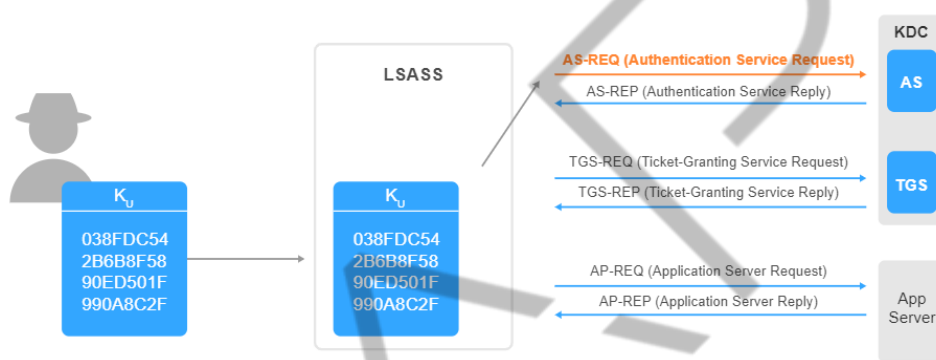


Figura 20 – Técnica Pass-the-Key
Fonte: Google Imagens (2019)

Apresentado na figura acima, como em uma subcategoria ou uma especialização, na técnica *Overpass-the-Hash* ou *Pass-the-Key*, o atacante obtém uma das chaves de longo prazo de um usuário, que são derivadas da senha dele, e a injeta em memória, no processo LSASS. Assim, quando a requisição AS-REQ é feita, essa chave será usada para criar um carimbo de tempo cifrado como autenticador.

No ataque *Pass-the-Ticket*, o invasor obtém um tíquete de serviço válido, gerado pelo AS ou TGS, em face de uma requisição do usuário. O tíquete obtido pode ser tanto um TGT ou tíquete de serviço. O tíquete é injetado na LSASS e usado para requisitar outros tíquetes ou acessar o serviço correspondente ao tíquete.

Apesar de amplamente conhecido na literatura como PtT, esse tipo de ataque, quando usando o TGT, mereceria um nome especial dentro das subcategorias de ataques PtT. Defende-se isso por ele se tratar de um tíquete especial quando comparado com os tíquetes de serviço e dar mais poder ao atacante: o poder de requisitar acesso a serviços em nome daquele usuário a quem o tíquete pertence.

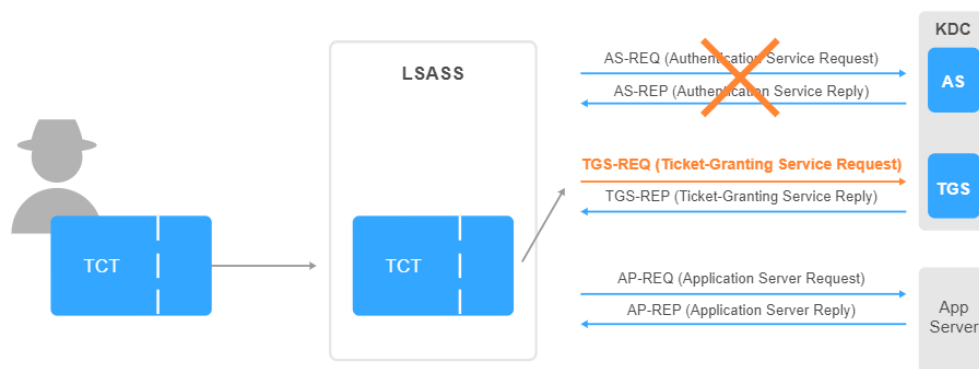


Figura 21 – Técnica Pass-the-Ticket
Fonte: Google Imagens (2019)

6.6 Segurança de Senhas no Linux

Toda conta de usuário em sistema Linux tem uma entrada no arquivo **/etc/passwd**. Esse arquivo contém entradas, uma linha por usuário, que especificam diversos atributos para cada conta. Cada entrada nesse arquivo tem o mesmo formato, com campos separados por "dois pontos":

usuário:senha:UID:GID:NomeReal:/diretório/pessoal:shell

Exemplo de arquivo **/etc/passwd**:

```
root:x:0:0:root:/root:/bin/bash
ftp:x:14:50:FTP                                User:/var/ftp:/sbin/nologin
postgres:x:26:26:PostgreSQL                    Server:/var/lib/pgsql:/bin/bash
fulano:x:500:500:Fulano                        da Silva:/home/fulano:/bin/bash
siclano:x:501:501:Siclano Souza:/home/siclano:/bin/bash
```

OBS.: "x": é a indicação de que essa conta possui hash da senha omitida no arquivo **/etc/shadow**.

O arquivo **/etc/shadow** armazena as senhas em um formato criptografado, com algumas informações adicionais relacionadas às senhas dos usuários. Contém uma entrada por linha, separada em campos, para cada usuário definido no **/etc/passwd**.

usuário:senha:última alteração:mínimo:máximo:aviso de
expiração:inativo:expiração:

Exemplo de arquivo **/etc/shadow**:

```
root:$6$utLIMf6gY40$Eoe.2M7uqTK1UBtnz3yz4T:14768:0:99999:7:  
daemon*:14715:0:99999:7::  
lp*:14715:0:99999:7::  
shutdown*:14715:0:99999:7::  
ftp*:14715:0:99999:7::  
nobody*:14715:0:99999:7::  
postgres:!:14855:::::::  
fulano:$6$nsd3oKxkg0$klEvBac347MZXYfOnrc56ybz:14768:0:99999:7::  
siclano:$6$AaNgoXJSPsv/$cupVfet0y4cIYLpPSWyNQM:14771:0:99999:7::  
beltrano:$6$Qc3AYPlBx0$ikGowHMwzkqRz9QGZzQfwfu:14770:0:99999:7::
```

A *string* completa com o hash, que representa a senha, é armazenada no arquivo **/etc/shadow** no formato **\$ID\$SALT\$SENHA**. O caractere "\$" (cifrão) delimita cada item da *string*.

```
nome_do_usuário:$6$eKn9QGMQ$Ofi1OoClVpw/cCbTsD4YUKgfurcQoBxsZ9Tlk5V  
BjTpAjSYFt.M9shPtQVnouNr4/3PRDP/eMqkoWQpuBxsRk1:15362:0:99999:7::
```

Sendo o \$6 o tipo de algoritmo de senha utilizado, conforme abaixo:

- \$1 = MD5.
- \$2 = Blowfish.
- \$2a = eksblowfish.
- \$5 = SHA-256.
- \$6 = SHA-512.

O \$eKn9QGMQ\$ é o *salt* (do inglês; sal) implementado no hash de senhas dos sistemas Linux de forma randômica para aumentar o nível de complexidade e

segurança do hash. O restante (em roxo no exemplo acima) seria a senha após ser calculada com o hash escolhido (SHA-512, neste caso).

Apesar de ser uma implementação mais segura do que nos ambientes Windows, existe uma forma de burlar essa implementação e se chegar na senha em claro do usuário.

A aplicação **unshadow**, que utiliza ataques do tipo dicionário, separa os campos da entrada do arquivo **/etc/shadow** comparado ao **/etc/passwd** e tenta comparar o hash (por meio de *password guessing* – técnica de comparação de hash) com as entradas do dicionário utilizado, chegando, assim, à senha em claro.

6.7 Wordlists

Os dicionários de senha, ou *wordlists* (também chamados simplesmente de “dicionários”), são arquivos em texto contendo possíveis senhas, uma em cada linha, para ataques do tipo força bruta (*brute force*), em que seriam usadas técnicas de *password guessing* e *password cracking*.

A diferença básica entre essas duas técnicas de *brute force* são bem claras. No *guessing*, o atacante que não possui o hash da senha legítima do usuário, realiza o ataque por meio de tentativas de login, ocorrendo o sucesso quando o hash da senha usada pelo atacante combina com o hash da senha legítima do usuário.

Já no *cracking*, o atacante explora vulnerabilidades da implementação do sistema de autenticação tentando logar-se com o hash da senha legítima do usuário, o qual foi capturado por outros métodos, em vez de utilizar uma senha propriamente dita.

Um exemplo claro de utilização da técnica de *password cracking* seria no ataque *Pass-the-Hash* em sistemas Windows, em que na posse do hash da senha do usuário, o atacante conseguirá realizar o login.

No caso de ataques força bruta, utilizando a técnica de *password guessing*, e sabendo que esse tipo de ataque consiste em tentativas, o sucesso do ataque está diretamente ligado ao dicionário utilizado.

Quanto mais eu souber do meu alvo, mais “palavras” eu poderei inserir em meu dicionário a fim de realizar um ataque bem-sucedido. Logo, para um ataque com maior probabilidade de sucesso, é necessário um trabalho bem-feito de Engenharia Social e Inteligência sobre meu alvo.

Existem algumas ferramentas no Kali que podem nos auxiliar na criação do nosso dicionário, tais como:

a) CUPP:

Uma ferramenta interativa, escrita em Python, amplamente usada para criação de *wordlists* por meio de análises de Engenharia Social. Um fator interessante sobre ela é que possibilita a inserção de caracteres especiais de forma automatizada nas senhas geradas.

Sua forma de uso é simples:

```
cupp -i
```

b) CeWL:

O CeWL é uma ferramenta interessante para a geração de *wordlists* por intermédio de análise de inteligência em sites. Basta passar à ferramenta um endereço que ela cria a *wordlist* com base nas palavras do site informado, desta forma:

cewl	<opções>	URL
cewl -m5 -d2 -w pqdt.txt	www.sejalivre.org	

Onde:

- m informa o tamanho mínimo da senha.
- d informa a quantidade de links que o CeWL irá acessar do site informado para capturar palavras.

c) CRUNCH:

O Crunch é uma das ferramentas mais famosas para a criação de *wordlists*. Com ela, é possível criar senhas randômicas, como senhas numéricas, por exemplo,

senhas baseadas em anagramas, senhas que possuam somente determinados caracteres etc.

Seu uso é, basicamente, assim:

```
crunch <min> <max> <opções> <caracteres> > <arquivo>

crunch 5 5 admin > dicionario.txt

crunch 1 1 -p abcde > dicionario.txt

crunch 1 1 -p ab cd ef > dicionario.txt

crunch 9 9 -f /usr/share/crunch/charset.lst lalpha -t @@admin@@ >
dicionario.txt
```

O Kali possui alguns dicionários já instalados que foram confeccionados baseados na Língua Inglesa ou em outros fatores, como, por exemplo, a lista de senhas mais usadas no último ano. Esses dicionários podem ser encontrados em:

```
/usr/share/john/password.lst

/usr/share/metasploit-framework/data/wordlists/password.lst

/usr/share/wordlists/rockyou.txt
```

Se você é jogador de CTF – *Capture de Flag* ou se interessa pelo assunto, saiba que a rockyou é a lista mais usada nesses *challenges* e, com certeza, sua *flag* poderá ser encontrada nela.

6.8 Rainbow tables

As *rainbow tables* são tabelas pré-computadas utilizadas para quebrar hashes de senhas. As tabelas têm o intuito de economizar tempo/processamento da máquina em troca de espaço em disco.

Suponhamos que temos uma função de hash de senha H e um conjunto finito de senhas P . O objetivo é pré-calculer uma estrutura de dados que, dada qualquer saída h da função hash, possa localizar um elemento p em P tal que $H(p) = h$ ou

determinar que não existe tal p em P . A maneira mais simples de fazer isso é calcular $H(p)$ para todo p em P , mas armazenar a tabela requer $\Theta(|P|n)$ bits de espaço, onde n é o tamanho de uma saída de H , que é proibitivo para grandes $|P|$.

As *rainbow tables* são uma técnica para diminuir esse requisito de espaço. A ideia é definir uma função de redução R que mapeie os valores de hash de volta para os valores em P . Observe, no entanto, que a função de redução não é realmente um inverso da função hash, mas uma função diferente com um domínio e um codomínio trocados. Ou seja, baseado em tabelas predefinidas de cálculos de hash para senhas já conhecidas, é possível chegar à senha legítima do usuário apenas juntando essas partes conhecidas dos hashes.

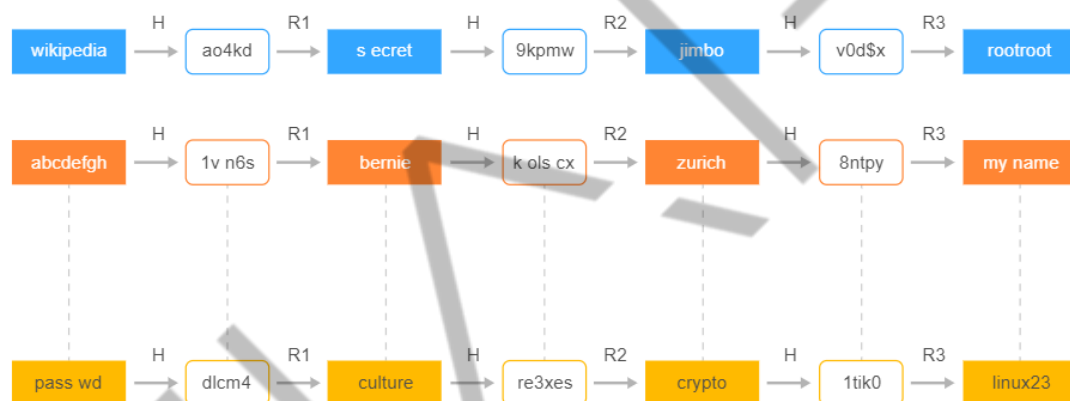


Figura 22 – Rainbow Tables
Fonte: Google Imagens (2019)

Essa é uma técnica amplamente usada para ataques em sistemas Windows e *rainbow tables* para versões específicas do Windows que podem ser baixadas livremente na Internet.

6.9 SMB: o vilão dos usuários

Para ser bem-sucedido em ataques do tipo *password cracking*, é necessário que se capture o hash da senha legítima do usuário. Apesar de poder ser realizado de diversas formas, a mais comum delas é pela exploração do protocolo SMB (Server Message Block – protocolo de rede da camada de aplicação usado para fornecer acesso compartilhado a arquivos, diretórios, impressoras e portas seriais).

O serviço de compartilhamento de arquivo, por padrão, é executado na porta 445. Se mal configurado, permitindo uma NULL SESSION, por exemplo, pode ser útil para enumerar usuários e compartilhamentos do sistema-alvo.

No Kali, existem algumas ferramentas para essa tarefa, tais como:

a) ENUM4LINUX:

Ferramenta específica para a enumeração por meio do protocolo SMB e pode ser usada assim:

```
enum4linux [options] ip  
  
enum4linux -U 192.168.0.33 → Enumera usuários  
enum4linux -S 192.168.0.33 → Enumera compartilhamentos
```

b) RPCCLIENT:

Ferramenta para a enumeração por meio do protocolo RPC (*Remote Procedure Call* – protocolo de comunicação entre processos que proporciona o acesso a processos em máquinas remotas). Assim como o SMB, o RPC possui vulnerabilidades que permitem a enumeração de usuários do sistema-alvo.

O rpcclient pode ser usado assim:

```
rpcclient -U <qualquer usuário> <ip>  
rpcclient -U Teste 192.168.0.33  
enumdousers → Usuários
```

c) NMAP:

Famosa ferramenta de enumeração e *information gathering*, por meio dos seus scripts o NMAP consegue enumerar usuários e compartilhamentos SMB desta forma:

```
nmap --script smb-enum-users.nse -p445 <ip> Usuários  
nmap --script smb-enum-shares.nse -p445 <ip> Compartilhamentos
```

7 FERRAMENTAS DE QUEBRA DE SENHAS

Nesta seção, iremos apresentar as principais ferramentas de quebra de senhas presentes no Kali Linux.

7.1 Hydra



Figura 23 – Hydra
Fonte: Google Imagens (2019)

O Hydra é uma ferramenta famosa que realiza ataques de dicionário a serviços. Seu uso é, basicamente, assim:

```
hydra -l <login> -p <senha> <IP> <serviço> -t <tasks>
```

Abaixo, um exemplo de uma tentativa única (-t 1) com o usuário root (-l root) e a senha 12345 (-p 12345) no endereço 10.0.2.2 no serviço ssh:

```
hydra -l root -p 12345 10.0.2.2 ssh -t 1
```

Podemos passar um dicionário de logins e senhas com o parâmetro -L e -P:

```
hydra -L <arquivo_login> -P <arquivo_senha> <IP> <serviço>
```

```
hydra -L login.txt -P senha.txt 10.0.2.2 ssh -t 4
```

O Hydra também possui uma interface gráfica que pode ser chamada com o comando:

```
xhydra
```

7.2 Medusa

Assim como o Hydra, o Medusa realiza ataques de dicionário a serviços. Seu uso é basicamente:

Para listar os módulos:

```
medusa -q | more
```

Comando:

```
medusa -h <ip> -u <usuario> -p <senha> -M <modulo>
```

```
medusa -h 10.10.100.25 -u admin -p 1234 -M ftp
```

Lista de IP, usuários e senhas

```
medusa -H <ips> -U <usuarios> -P <senhas> -M <modulo>
```

```
medusa -H hosts.txt -U users.txt -P pass.txt -M smbnt
```

7.3 John the Ripper



Figura 24 – John the Ripper

Fonte: Google (2019)

O *John the Ripper* (o nome da ferramenta faz alusão a um famoso psicopata do século 19, Jack, o estripador) é uma das ferramentas mais usadas em pentest, sendo ele um utilitário que faz quebra de senhas de três modos:

- **WordList** → Ele tenta por uma *wordlist*, testando as combinações de senha/usuário.
- **Single Crack** → Ele tenta quebrar a senha usando as informações de login.
- **Incremental** → Sendo o modo mais robusto no John the Ripper, ele tentará cada caractere possível até achar a senha correta. E por esse motivo, é indicado o uso de parâmetros com o intuito de reduzir o tempo de quebra.

No caso do modo incremental, é necessário ter o arquivo contendo o hash da senha do usuário (SAM no Windows ou /etc/shadow no Linux).

Seu uso é basicamente:

```
john <arquivo> --wordlist=dicionario.txt  
john <arquivo> --single  
john <arquivo> --incremental  
john <arquivo> --wordlist=dicionario.txt --format=NT
```

As senhas quebradas são armazenadas no arquivo:

```
<home_do_usuario>/.john/john.pot
```

E assim como o Hydra, o JtR também tem uma Interface Gráfica, que pode ser chamada pelo comando:

```
johnny
```

7.4 OphCrack

O OphCrack é uma ferramenta nativa do Kali Linux e específica para uso com Rainbow Tables. Com uma interface gráfica bem intuitiva, o OphCrack é ideal para realizar ataques de *Brute Force* em sistemas Windows.

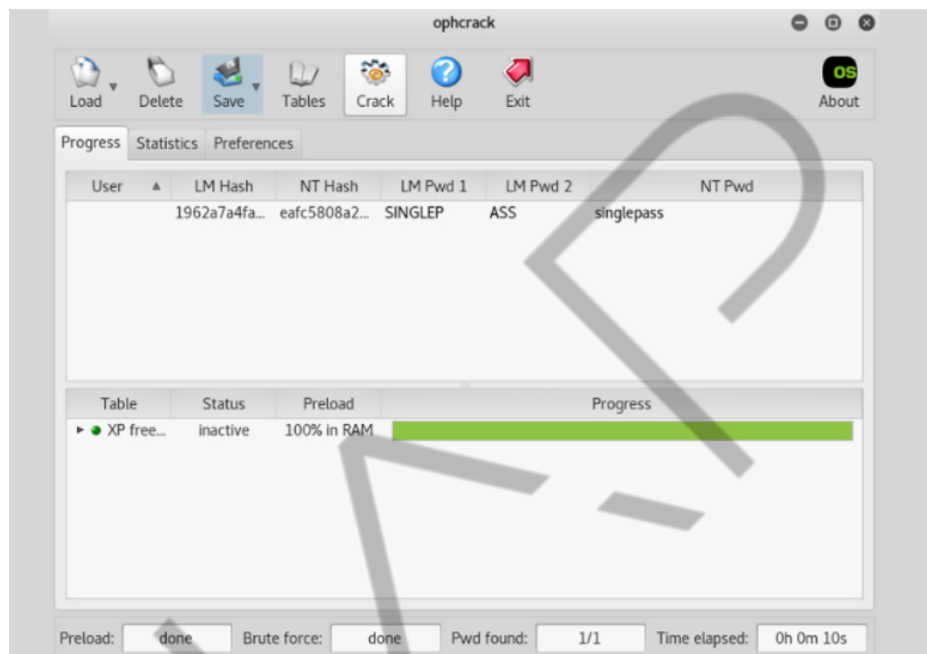


Figura 25 – OphCrack
Fonte: Google Imagens (2019)

7.5 Hashcat

Também nativo do Kali, o Hashcat é uma ferramenta específica para realizar ataques do tipo *Password Cracking*. Uma das *features* mais interessantes dela é a possibilidade de se utilizar o processador da placa de vídeo para dar mais performance ao ataque.

Seu uso é basicamente:

```
hashcat -m <tipo do hash> -a <tipo do ataque> -o <arquivo de saída> <arquivo de hash> <dicionário>
```

Exemplo:

```
hashcat -m 1800 -a 0 -o cracked.txt /etc/shadow /usr/share
```

```
/usr/share/wordlists/rockyou.txt
```

Caso os drivers da placa de vídeo não estejam instalados, é necessário utilizar a opção `--force`, desta forma:

```
hashcat -m 1800 -a 0 -o cracked.txt /etc/shadow /usr/share  
/usr/share/wordlists/rockyou.txt --force
```

7.6 Metasploit Framework

O Metasploit (MSF) é um framework criado por H.D.Moore, que serve para elaboração e execução de um repositório de *exploits*.



Figura 26 – Metasploit Framework

Fonte: Google (2019)

Apesar de não ser esse o objetivo da ferramenta, o MSF possui módulos de ataque força bruta para diversas aplicações, por exemplo, o SSH, que pode ser executado por meio da sequência de comandos abaixo:

```
service postgresql start  
  
msfconsole  
  
use <auxiliar>  
  
auxiliary/scanner/ssh/ssh_login  
  
auxiliary/scanner/ftp/ftp_login
```

```
auxiliary/scanner/smb/smb_login  
auxiliary/scanner/telnet/telnet_login  
show options  
set <parametros>  
run
```

7.7 LAB

Você, analista de segurança da VortSec Solutions, empresa especializada na realização de testes de segurança em aplicações web, foi selecionado para participar do pentest do sistema de um dos clientes. Como escopo do pentest, as informações abaixo foram passadas pelo gerente do projeto:

- URL do alvo: attackdirect.samsclass.info
- Dicionário: rockyou.txt
- Usuário da aplicação: root

Sua tarefa será realizar um *brute force* em um formulário de login no site do cliente, utilizando o Hydra. Para tal, utilize sua máquina virtual do Kali Linux.

Após o carregamento do sistema, abra um terminal e siga os passos abaixo:

- a) Primeiro, iremos descompactar a *wordlist* rockyou em seu Kali:

```
cd /usr/share/wordlists  
gzip -d rockyou.txt.gz
```

- b) Feito isso, podemos iniciar o ataque. Execute o comando abaixo para testar a aplicação utilizando o Hydra:

```
hydra -l root -P rockyou.txt attackdirect.samsclass.info -t 1 http-get
```

Qual foi o resultado? Conseguiu quebrar a senha?

Essa é uma aplicação vulnerável que aceita vários parâmetros como senha válida. Caso a saída do seu Hydra tenha sido parecida com a imagem abaixo, parabéns, você conseguiu quebrar a senha do usuário root.

```
root@kali:~/brute# hydra -l root -p password attack.samsclass.info http-get /brute0/
Hydra v7.4.2 (c)2012 by van Hauser/THC & David Maciejak - for legal purposes only

Hydra (http://www.thc.org/thc-hydra) starting at 2013-09-06 14:14:38
[DATA] 1 task, 1 server, 1 login try (l:1/p:1) ~1 try per task
[DATA] attacking service http-get on port 80
[80][www] host: 199.188.72.153 login: root password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2013-09-06 14:14:42
root@kali:~/brute#
```

Figura 27 – Saída do Hydra no Kali Linux
Fonte: Elaborado pelo autor (2019)

REFERÊNCIAS

AZEVEDO, E.; FAVERI, J. G.; NUNES, S. E. Esteganografia. **Rev. Cienc. Exatas Tecnol.**, v. 10, n. 10, p. 31-35, 2015. Disponível em: < [Artigo Científico Estagnografia - Segurança da Tecnologia da Informação](#) >. Acesso em: 28 nov. 2024.

MARTINS, H. P. *et al.* Técnicas de Segurança Aplicação de Esteganografia em Imagens. **Caderno de Estudos Tecnológicos**, v. 5, n. 1, 2017.

MORAES, F. L. da R. **Honeyhashes e Honeytickets**: detecção de ataques pass-the-hash e pass-the-ticket em redes de domínio Active Directory do Windows. Dissertação (Mestrado em Engenharia Elétrica). Universidade de Brasília, Brasília, 2016. Disponível em: <[Repositório Institucional da UnB: Honeyhashes e honeytickets : detecção de ataques pass-the-hash e pass-the-ticket em redes de domínio active directory do Windows](#)>. Acesso em: 28 nov. 2024.

OLIVEIRA, R. R. **Criptografia simétrica e assimétrica**: os principais algoritmos de cifragem. 2012. Disponível em: <<http://www.ronielton.eti.br/publicacoes/artigorevistasegurancadigital2012.pdf>>. Acesso em: 28 nov. 2024.

SERAFIM, V. da S. **Introdução à criptografia**: funções criptográficas de Hash. 2012. Disponível em: <http://www.serafim.eti.br/academia/recursos/Roteiro_08-Funcoes_de_Hash.pdf>. Acesso em: 28 nov. 2024.

TAVARES, P. H. N. **Estudo e implementação de algoritmos de resumo (hash) criptográfico na plataforma Intel® XScale®**. Trabalho final (Mestrado profissional em Computação). Universidade Estadual de Campinas, Campinas, SP, 2006. Disponível em: <[Terminal RI - Sophia Biblioteca Web](#)>. Acesso em: 28 nov. 2024.

VICENTINI, F. R. S.; OLIVEIRA, H. C.; GODOY, M. A.; MARTINS, H. P. **Técnicas de Segurança: Aplicação de Esteganografia em Imagens**. Caderno de Estudos Tecnológicos, v. 5, n. 1, p. 1-18, 2017. Disponível em: < [Técnicas de Segurança Aplicação de Esteganografia em Imagens | Semantic Scholar](#) >. Acesso em: 28 nov. 2024.